

RELATÓRIO TESTES AUTOMATIZADOS E ANÁLISE DE MUTAÇÃO

Guilherme José da Silva

Contextualização do Sistema Avaliado

O sistema-alvo desta atividade é o Scholarship Eligibility Evaluator, um avaliador de elegibilidade para bolsas de estudo. O sistema recebe cinco entradas:

- **Idade** (age)
- **Coeficiente de Rendimento** (gpa)
- **Taxa de presença** (attendance_rate)
- **Cursos obrigatórios concluídos** (has_required_courses)
- **Histórico disciplinar** (disciplinary_record)

Com base nessas entradas, o sistema retorna um dos três resultados possíveis:

- **APPROVED** – candidato atende a todos os requisitos.
- **REJECTED** – candidato reprovado por um ou mais motivos.
- **MANUAL_REVIEW** – candidato elegível para revisão manual.

Além do status, o sistema retorna uma lista de motivos (reasons) que compõem a saída observável e são verificados em todos os testes.

1. Escolhas Tecnológicas

O ambiente tecnológico foi construído com as seguintes ferramentas:

- **Linguagem:** Python. Escolhida por possuir amplo suporte a ferramentas de testes e permitir a rápida implementação.
- **Framework de Testes:** Pytest. Justifica-se pela simplicidade em criar casos de testes unitários através da função padrão assert e pela facilidade em validar exceções.
- **Ferramenta de Mutação:** Cosmic Ray. Adotada por ser uma biblioteca madura em Python para análise de mutação, fornecendo relatórios granulares de sobrevivência.

2. Projeto de Testes Funcionais e Estruturais

Pergunta 1: Quais classes de equivalência foram identificadas?

A especificação do avaliador mapeia as entradas para três saídas primárias. As classes de equivalência derivadas foram:

- **Entradas Válidas (Caminho Feliz):** Candidatos com idade ≥ 18 , GPA entre 7.0 e 10.0, presença $\geq 80\%$, disciplinas requeridas concluídas e sem histórico disciplinar.
- **Entradas de Revisão:** Candidatos que possuem idade entre 16 e 17, ou GPA entre 6.0 e 6.9, ou presença entre 75% e 79.9%.
- **Entradas de Rejeição:** Idade < 16 , GPA < 6.0 , Presença $< 75\%$, disciplinas requeridas ausentes ou histórico disciplinar presente.
- **Entradas Inválidas (Tratamento de Erros):** Notas menores que 0.0 ou maiores que 10.0; taxas de presença menores que 0.0 ou maiores que 100.0.

Pergunta 2: Quais valores limite foram considerados?

A técnica de análise de valor limite focou em testar as transições de status da regra de negócio:

- **Idade:** 16 (transição de Reject para Review), 17 (fronteira entre Reject e Review), 18 (limite para aprovação automática).
- **GPA:** 6.0 (fronteira entre Reject e Review), 7.0 (fronteira entre Review e Approve), 10.0 (limite superior válido).
- **Presença:** 75.0 (fronteira de transição de rejeição), 80.0 (limite para aprovação automática).
- **Valores Inválidos:** 11.0 para GPA e 105.0 para presença para disparar exceções (ValueError).

Pergunta 3: Quais decisões importantes do código foram testadas?

Os testes cobriram ramificações essenciais de tomada de decisão. A tabela abaixo mapeia cada decisão/branch do código aos testes correspondentes:

Decisão / Condição no Código	Caso(s) de Teste Associado(s)
if gpa < 0.0 or gpa > 10.0	test_evaluate_scholarship_invalid_gpa
if attendance_rate < 0.0 or attendance_rate > 100.0	test_evaluate_scholarship_invalid_attendance
if age < 16	test_evaluate_scholarship_rejected_age
elif age <= 17	test_evaluate_scholarship_manual_review_age, test_evaluate_scholarship_boundary_age_16
if gpa < 6.0	test_evaluate_scholarship_rejected_gpa
elif gpa < 7.0	test_evaluate_scholarship_boundary_gpa_6_0
if attendance_rate < 75.0	test_evaluate_scholarship_boundary_attendance_75_0 (fronteira) test_evaluate_scholarship_rejected_attendance (fronteira)
elif attendance_rate < 80.0	test_evaluate_scholarship_boundary_attendance_75_0
if not has_required_courses	test_evaluate_scholarship_rejected_required_courses
if disciplinary_record	test_evaluate_scholarship_rejected_disciplinary
if rejection_reasons:	test_evaluate_scholarship_rejected_age, test_evaluate_scholarship_rejected_gpa, test_evaluate_scholarship_rejected_disciplinary
if review_reasons:	test_evaluate_scholarship_manual_review_age, test_evaluate_scholarship_boundary_age_16, test_evaluate_scholarship_boundary_gpa_6_0, test_evaluate_scholarship_boundary_attendance_75_0
Retorno APPROVED	test_evaluate_scholarship_approved, test_evaluate_scholarship_boundary_attendance_80_0, test_evaluate_scholarship_boundary_gpa_10_0, test_evaluate_scholarship_boundary_gpa_7_0

Todos os casos de teste realizam asserções compostas, validando tanto o status (APPROVED, REJECTED, MANUAL_REVIEW) quanto a correspondência exata dos itens na lista de motivos (reasons) retornada pela função.

3. Análise de Mutação e Adequação

Pergunta 4: Qual foi o score de mutação inicial e final? O que explica a diferença?

O score de mutação inicial identificou 34 mutantes sobreviventes de um total de 100 injetados (Score: 66%). Após a análise, foram incorporados dois novos testes:

- test_evaluate_scholarship_boundary_gpa_10_0
- test_evaluate_scholarship_boundary_gpa_7_0

Esses acréscimos visaram matar mutantes operacionais de limite, como Gt_GtE e Lt_LtE. Com isso, a segunda rodada da ferramenta reportou 28 mutantes sobreviventes (Score: 72%).

A diferença demonstra que a cobertura original não avaliou exaustivamente as igualdades restritas no limiar de notas. Os dois testes adicionados mataram especificamente os mutantes:

- **core/ReplaceComparisonOperator_Gt_GtE (índices 0 e 1):** que alteravam > para >= na validação de GPA máximo.
- **core/ReplaceComparisonOperator_Lt_LtE (índices 5 e 6):** que alteravam < para <= na fronteira de GPA 7.0.

Pergunta 5: A cobertura de código e o score de mutação apontaram para as mesmas lacunas? Se divergiram, por quê?

As métricas divergiram. Para quantificar essa divergência, foi executado o plugin pytest-cov sobre a suíte inicial:

- **Cobertura de Statements (linhas) no módulo principal:** 95% (2 linhas não cobertas).
- **Cobertura de Branches:** 94% no total do projeto (3 branches parcialmente cobertos no módulo principal).
- **Linhas não cobertas:** 84-91 (sendo as linhas 84-91 referentes ao bloco if `__name__ == '__main__':`, que não é executado pelos testes unitários).

Apesar da alta cobertura de linhas (95%), o score de mutação inicial foi de apenas 66%. Isso evidencia que executar a maioria das linhas não garante que o comportamento lógico esteja corretamente validado. Por exemplo, a suíte passava em todas as linhas executadas da função principal, mas a análise de mutação revelou que trocar `<= 17` por `< 17` ainda permitiria a passagem dos testes.

A mutação questiona a robustez do teste, enquanto a cobertura clássica apenas confirma se o código foi lido. Enquanto a cobertura de branches foi de 94%, a mutação revelou que mesmo branches cobertos podem não ter suas fronteiras adequadamente testadas.

Pergunta 6: Quais mutantes você classificou como equivalentes, e com base em que argumento?

Mutante equivalente identificado:

- **ID/Operador:** core/ReplaceComparisonOperator_Eq_NotEq 0
- **Trecho original:** if `__name__ == '__main__':`
- **Trecho alterado:** if `__name__ != '__main__':`
- **Justificativa:** Este mutante alterou a declaração do módulo de if `__name__ == '__main__':` para if `__name__ != '__main__':`. Este trecho é executado apenas quando o script é chamado diretamente no terminal para demonstração, não fazendo parte das funções de negócio testáveis. Como os testes unitários (via Pytest) apenas importam a função, alterar essa condicional não provoca nenhuma mudança no comportamento observável do sistema sendo testado. Nenhum teste seria capaz de matá-lo, pois o código mutado nunca é executado no contexto de teste.

Lacunas Encontradas na Suíte (Mutantes Sobreviventes Analisados)

Além do mutante equivalente, a análise revelou vulnerabilidades na suíte inicial que explicam os demais sobreviventes. Abaixo está a análise padronizada dos três mutantes sobreviventes examinados individualmente:

1. **core/ReplaceComparisonOperator_Gt_GtE (índice 0)**
 - **Trecho original:** if `gpa > 10.0:`
 - **Trecho alterado:** if `gpa >= 10.0:`
 - **Por que sobreviveu:** A suíte não testava o valor exato de GPA = 10.0; o mutante faz com que 10.0 seja rejeitado, mas nenhum teste verificava esse limite superior.
 - **Teste criado para matá-lo:** `test_evaluate_scholarship_boundary_gpa_10_0`
2. **core/ReplaceComparisonOperator_Lt_LtE (índice 5)**
 - **Trecho original:** elif `gpa < 7.0:`
 - **Trecho alterado:** elif `gpa <= 7.0:`
 - **Por que sobreviveu:** A suíte não testava o valor exato de GPA = 7.0; o mutante faz com que 7.0 caia

em revisão em vez de aprovação.

- **Teste criado para matá-lo:** `test_evaluate_scholarship_boundary_gpa_7_0`

3. **core/ReplaceComparisonOperator_Gt_GtE (índice 1)**

- **Trecho original:** `if attendance_rate > 100.0:`
- **Trecho alterado:** `if attendance_rate >= 100.0:`
- **Por que sobreviveu:** Na suíte inicial, nenhum teste enviava exatamente `attendance_rate=100.0`. Logo, o mutante transformava a validação de "maior que 100" para "maior ou igual a 100" sem que nenhum teste falhasse ou caísse na exceção `ValueError`.
- **Teste criado para matá-lo:** O novo teste `test_evaluate_scholarship_boundary_gpa_10_0` envia `attendance_rate=100.0`. Ao rodar com o mutante, este teste disparou o erro de validação, não passou, e matou o mutante.

4. Análise de Adequação

Pergunta 7: A suíte pode ser considerada adequada? Por quê?

A suíte é considerada **parcialmente adequada**.

- **Funcionalmente:** cobre todos os caminhos primários definidos pela especificação, avaliando corretamente as respostas observáveis, o que inclui a correspondência exata dos itens na lista de motivos (reasons) e não apenas o status retornado.
- **Estruturalmente:** a suíte evoluiu para cobrir satisfatoriamente a avaliação isolada de atributos booleanos (como a falta de cursos obrigatórios) e limites inferiores das entradas. A cobertura de linhas do módulo principal é alta (95%). No entanto, a suíte apresenta limitações em testes combinados. Não há uma exaustão de cenários onde o candidato falha em múltiplos critérios simultaneamente (ex: idade muito baixa e histórico disciplinar ativo ao mesmo tempo).

O score de mutação final de 72% evidencia essa adequação parcial. Embora a suíte consiga matar mutantes que alteram as fronteiras diretas de decisão (como GPA e taxa de presença), o fato de ainda restarem mutantes sobreviventes indica que validações de combinações de regras, tipos de dados estritos ou lógicas internas mais complexas necessitariam de testes ainda mais rígidos e combinados para uma adequação total.

Pergunta 8: Que tipos de defeito nenhuma das duas métricas seria capaz de revelar?

Nenhuma das duas métricas é capaz de revelar:

- Defeitos de omissão ou interpretação de requisitos (quando a regra implementada não reflete a regra de negócio correta exigida pelo cliente).
- Falhas de integração com outros sistemas.
- Vazamentos de memória e gargalos de desempenho.
- Problemas de usabilidade ou experiência do usuário.
- Condições de corrida e problemas de concorrência.

Além disso, testes unitários tradicionais não revelam falhas que só emergem em configurações específicas de ambiente ou sequências de operações não previstas.

5. Considerações Finais

A atividade permitiu aplicar técnicas funcionais e estruturais de geração de testes, análise de adequação e análise de mutação sobre o sistema `Scholarship Eligibility Evaluator`. Os resultados evidenciam que:

- A cobertura de código, embora alta (95% de linhas), não garante a ausência de defeitos.
- A análise de mutação revelou lacunas específicas em valores de fronteira que a cobertura tradicional não detectou.
- A suíte final, embora melhorada, ainda apresenta limitações, especialmente na validação de combinações de condições booleanas.

O score de mutação não é comparável entre ferramentas de linguagens diferentes e não foi usado isoladamente como critério de nota. O que se avaliou foi a qualidade da análise dos mutantes sobreviventes, que permitiu identificar mutantes equivalentes, lacunas da suíte e direcionar a criação de testes mais robustos.